

PAVAN KUMAR A

Software Engineering Portfolio

A collection of production-grade distributed systems and AI-native engineering tools — spanning event-sourced microservices, LLM-powered observability agents, and real-time anomaly detection, built with the same rigor and design discipline applied to systems serving millions of users.

PROFESSIONAL SUMMARY

Senior Software Engineer with 7+ years architecting high-availability distributed systems in fintech and automotive sectors. Technical SME at Mastercard for consumer-facing financial microservices — improved core availability from 99% to 99.9999% (six nines), built SENTINEL (a production AI agent mining Splunk & Dynatrace traces), and delivered a team-wide AI agent library cutting design & rework cycle time by ~60%. Public GitHub projects demonstrate the full depth of these patterns in clean, portable, ADR-backed code.

CORE TOOLS & SKILLS

Java 17 • Python • Spring Boot • Apache Kafka • Axon Framework • AI/LLM Engineering

CQRS • Event Sourcing • Saga Pattern • Redis • PostgreSQL • Domain-Driven Design • Microservices • Contract Testing (Pact V3) • Testcontainers • FastAPI • MCP Servers • Anthropic Claude API • Prometheus • Grafana • AWS (S3, Lambda, SQS) • GCP • Azure • Docker • Kubernetes • CI/CD (GitHub Actions, Jenkins) • TypeScript • React

FEATURED PROJECTS

Distributed Systems — Tickera: Event-Driven Ticket Booking with CQRS, Saga & Pact Contract Testing (Java • Axon • Kafka)

AI Engineering — LogSense: LLM-Powered Log Ingestion, Clustering & Root Cause Analysis Agent (Python • Claude • MCP)

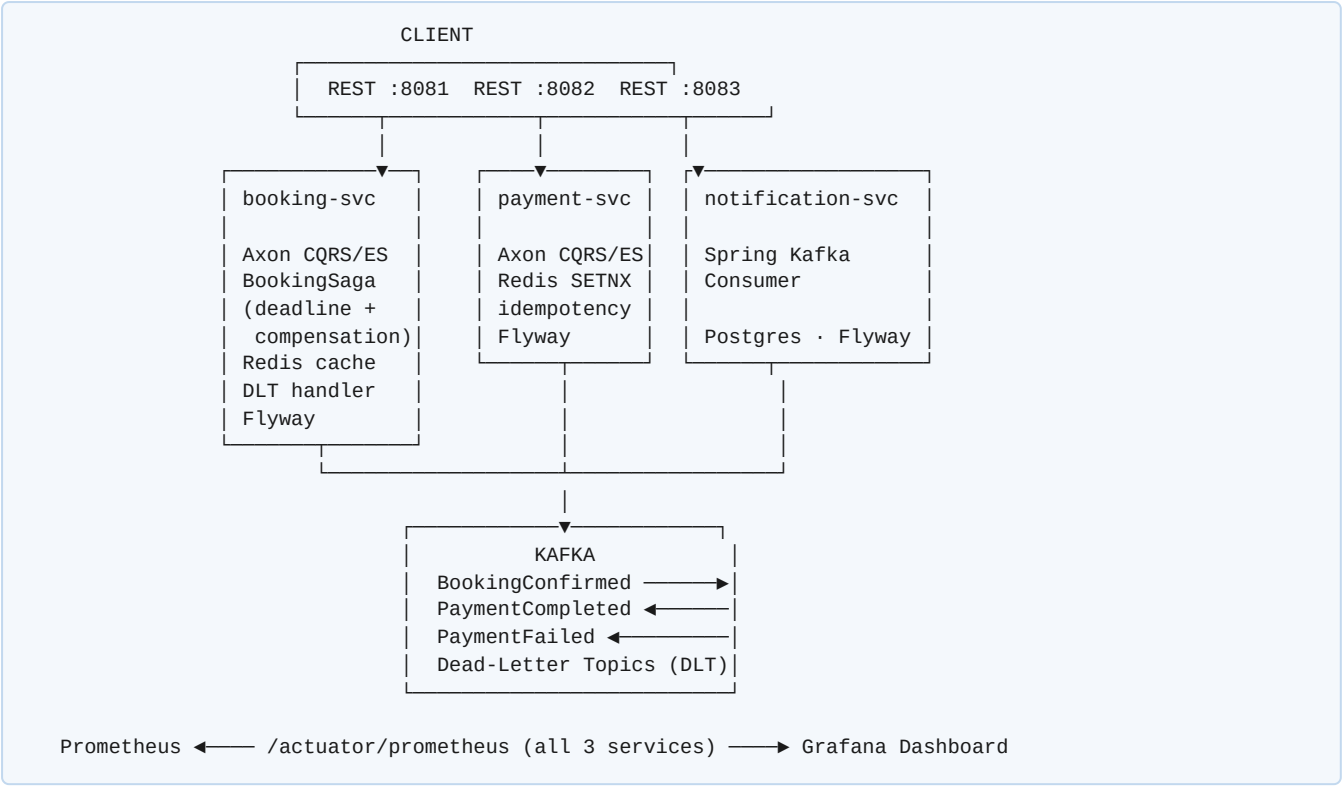
FinTech Observability — TxSentry: Real-Time Transaction Anomaly Scoring with Explainability (FastAPI • Prometheus)

Production AI — SENTINEL: Splunk + Dynatrace AI Risk Analyzer & Auto-Jira Story Generator (Mastercard, Python)

AI Acceleration — Team AI Agent Library: RCA, LLD Auto-Generation, PR Review & Test Generation (Mastercard)

Tickera — Event-Driven Ticket Booking

JAVA 17 · SPRING BOOT 3.2 · AXON 4.9 · APACHE KAFKA · REDIS · POSTGRESQL · DISTRIBUTED SYSTEMS
Repository: github.com/pavankumarale-hub/tickera



Tickera system architecture — CQRS/ES per service, Kafka integration backbone, Saga compensation, Redis idempotency

Brief

A production-shaped microservices system for booking event tickets, built to demonstrate senior distributed-systems patterns: **CQRS + Event Sourcing (Axon)** inside each service, **Kafka** as the integration backbone, a **Saga** for cross-service orchestration with timeout compensation, **Redis** for exactly-once idempotency and read-through caching, **Pact V3 consumer-driven contract testing** on the async Kafka boundary, and **Micrometer** → **Prometheus** → **Grafana** observability. One command to run everything; Architecture Decision Records document every major design choice.

System Metrics

3 Independent microservices — database-per-service isolation with Flyway versioning	15 min Saga deadline — compensation auto-fires if PaymentCompleted never arrives	3-layer Test strategy — unit (Axon), contract (Pact V3 async CDC), integration (Testcontainers)
Redis SETNX idempotency guard — exactly-once payment processing under at-least-once Kafka delivery	3-job CI Parallel GitHub Actions — unit + contract + integration fan out after compile gate → SonarQube	4 ADRs Architecture Decision Records — Axon vs plain Kafka, DB-per-service, idempotency, eventual consistency

Tools Used

Java 17, Spring Boot 3.2, Axon Framework 4.9, Apache Kafka, Redis (Jedis + Spring Data), PostgreSQL, Flyway, Pact V3 (async CDC), Testcontainers, JUnit 5, Micrometer, Prometheus, Grafana, Docker Compose, GitHub Actions, SonarQube.

Tickera — Engineering Depth

TESTING STRATEGY · OBSERVABILITY · SAGA COMPENSATION · EVENT SOURCING

Testing Strategy

- **Unit (Axon given/when/then)** — aggregate invariants and state transitions verified without Spring context or infrastructure. `BookingAggregateTest`, `PaymentAggregateTest`.
- **Contract (Pact V3 async CDC)** — booking-service publishes a consumer pact for `BookingConfirmed`; payment-service verifies it as provider. Committed pact files caught schema drift before integration.
- **Integration (Testcontainers)** — real Kafka + PostgreSQL spun per test run: confirms command → event → projection chain and that the integration event physically lands on the topic.
- **CI structure** — three parallel jobs fan out after a compile gate, then SonarQube aggregates all JaCoCo reports.

Observability Dashboard

- **HTTP Traffic row** — request rate (req/s), p95 latency (s), 5xx error rate per service.
- **Booking Funnel row** — Micrometer counters tracking `CREATED` → `CONFIRMED` → `PAID` / `CANCELLED` state transitions; Redis idempotency guard counter (first-delivery vs. duplicates blocked).
- **Infrastructure row** — Kafka consumer record rate, JVM heap, GC pause rate.
- **\$service variable** — single Grafana template variable filters all panels to one service or shows aggregate view.

Key Engineering Decisions

- **Axon over plain Kafka consumers** — Axon's aggregate pattern enforces command validation and event replay before a single line of Kafka consumer code exists; gives free event-sourced audit history.
- **Database-per-service** — each service owns its schema; cross-service joins go through Kafka events, not SQL. Enables independent schema evolution and deployment.
- **Redis SETNX for idempotency** — Kafka's at-least-once guarantee means payment-service may receive `BookingConfirmed` twice. `SETNX paymentId` ensures only the first delivery charges the card.
- **Saga with Axon deadline** — if `PaymentCompleted` never arrives within 15 minutes (e.g., payment-service down), the deadline handler fires a compensating `CancelBooking` command — preventing stuck bookings from holding inventory indefinitely.
- **Dead-Letter Topics + ExponentialBackOff** — transient failures retry with back-off; permanently failed events route to a DLT for manual inspection rather than blocking the partition.

"Most portfolios show CRUD over REST. This one shows the harder, more senior parts: event sourcing, sagas, eventual consistency, idempotency under at-least-once delivery, and contract testing on message boundaries — with the reasoning written down as Architecture Decision Records."

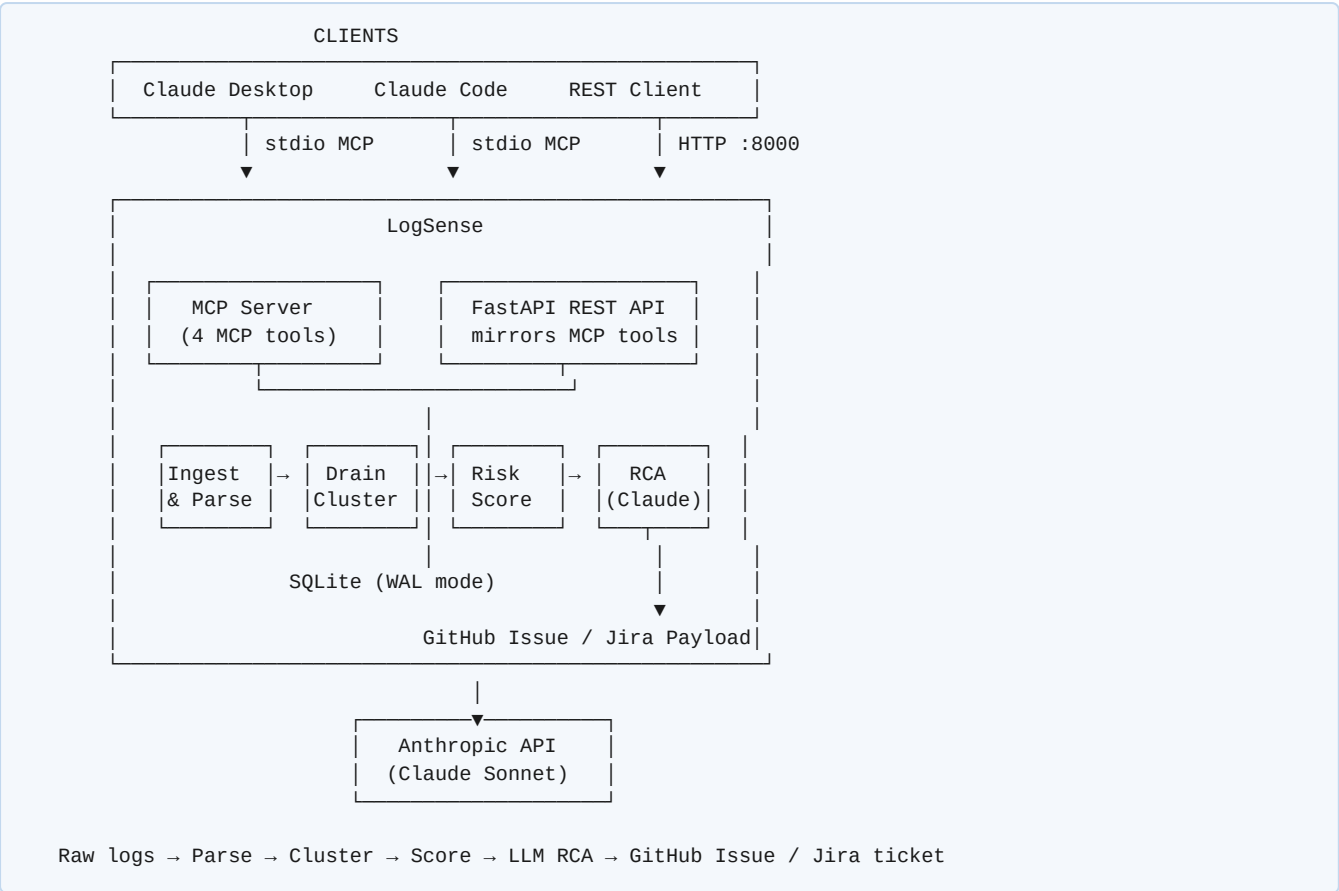
Repository Layout

```
tickera/
├─ common-events/      # shared Kafka integration-event contracts
├─ booking-service/    # Axon CQRS/ES, saga, Flyway, DLT, Redis cache
├─ payment-service/    # Axon CQRS/ES, Redis SETNX idempotency, Flyway
├─ notification-service/ # plain Spring Kafka consumer, Flyway, Postgres
├─ docs/               # architecture.md, LLD-booking-service.md, ADRs
├─ infra/              # Prometheus config, Grafana provisioning + dashboard JSON
├─ pacts/              # generated Pact V3 consumer contracts (committed)
├─ scripts/demo.sh     # end-to-end demo: happy path + failure/compensation
├─ Makefile            # build / test / demo shortcuts
├─ docker-compose.yml  # one-command full stack
└─ .github/workflows/ci.yml # compile → unit || contract || integration → sonar
```

LogSense — AI Log RCA Agent

PYTHON 3.11 · FASTAPI · ANTHROPIC CLAUDE API · MCP SERVER · DRAIN ALGORITHM · SQLITE · AI ENGINEERING

Repository: github.com/pavankumarale-hub/logsense



LogSense pipeline — Drain clustering surfaces error templates; Claude generates plain-language RCA with concrete next actions

Brief

LogSense ingests raw application logs (Spring Boot, Nginx, JSON, syslog), clusters similar errors using the **Drain template-extraction algorithm**, assigns a composite risk score to each cluster, and calls **Claude (Anthropic SDK)** to generate a plain-language root cause hypothesis with a concrete next action. Results surface as structured GitHub issues, Jira payloads, or live tool responses inside **Claude Desktop / Claude Code via an MCP server** — enabling an AI agent to diagnose and escalate incidents autonomously. Built as a legally-clean, public demonstration of the production log-mining and LLM-RCA patterns applied at Mastercard scale.

System Metrics

4 MCP tools — ingest_logs, get_clusters, get_rca_for_cluster, draft_incident_report	4 Log formats parsed — Spring Boot, Nginx, JSON structured, syslog	Drain Zero-dependency, deterministic log clustering — no LLM needed to identify templates
Risk Composite score: frequency × recency × severity — ranks clusters for prioritized triage	Dual Interfaces — FastAPI REST (:8000) + MCP stdio transport for AI-native clients	3 ADRs Clustering approach, LLM prompt + confidence design, MCP vs REST as primary interface

Tools Used

Python 3.11+, FastAPI, uvicorn, MCP Python SDK (FastMCP), Anthropic SDK (Claude), aiosqlite, Drain algorithm (custom implementation), pytest (unit + integration with cassette mocking), Docker Compose, GitHub Actions (matrix: Python 3.11 + 3.12).

LogSense — Engineering Decisions & Key Insights

LLM PROMPT DESIGN · CONFIDENCE SCORING · AI-NATIVE ARCHITECTURE · MCP PROTOCOL

MCP Tool Interface

- **ingest_logs** — accepts raw log content (any supported format); returns parsed + normalized entries and initial Drain-clustered templates stored to SQLite.
- **get_clusters** — returns error clusters ranked by composite risk score; supports `min_risk_score` filter for triage focus.
- **get_rca_for_cluster** — triggers LLM call for a specific cluster; returns hypothesis, confidence score (0–1), and recommended next action.
- **draft_incident_report** — builds a GitHub issue body or Jira payload in dry-run mode; one config flag to ship real tickets.

Key Engineering Decisions

- **Drain over embeddings for clustering** (ADR-0001) — Drain extracts log templates deterministically in linear time. Embedding-based clustering adds LLM latency and cost to every ingest call; Drain reserves the LLM budget for the high-value RCA step where it earns its keep.
- **Calibrated confidence scoring** (ADR-0002) — the LLM prompt asks for an explicit 0–1 confidence float before the hypothesis text, not as a second "grader" call. Single-pass JSON output keeps latency low and makes confidence a first-class assertable field in tests.
- **MCP as primary interface** (ADR-0003) — a REST API is easy to build but forces callers to orchestrate multi-step workflows manually. Exposing the pipeline as MCP tools lets Claude Desktop or Claude Code drive the full ingest → triage → RCA → draft-issue workflow in a single conversation turn, with no glue code.
- **SQLite with WAL mode** — zero-config storage that supports concurrent reads without blocking writes; one environment variable switches to PostgreSQL for multi-instance deployments.

Key Insights

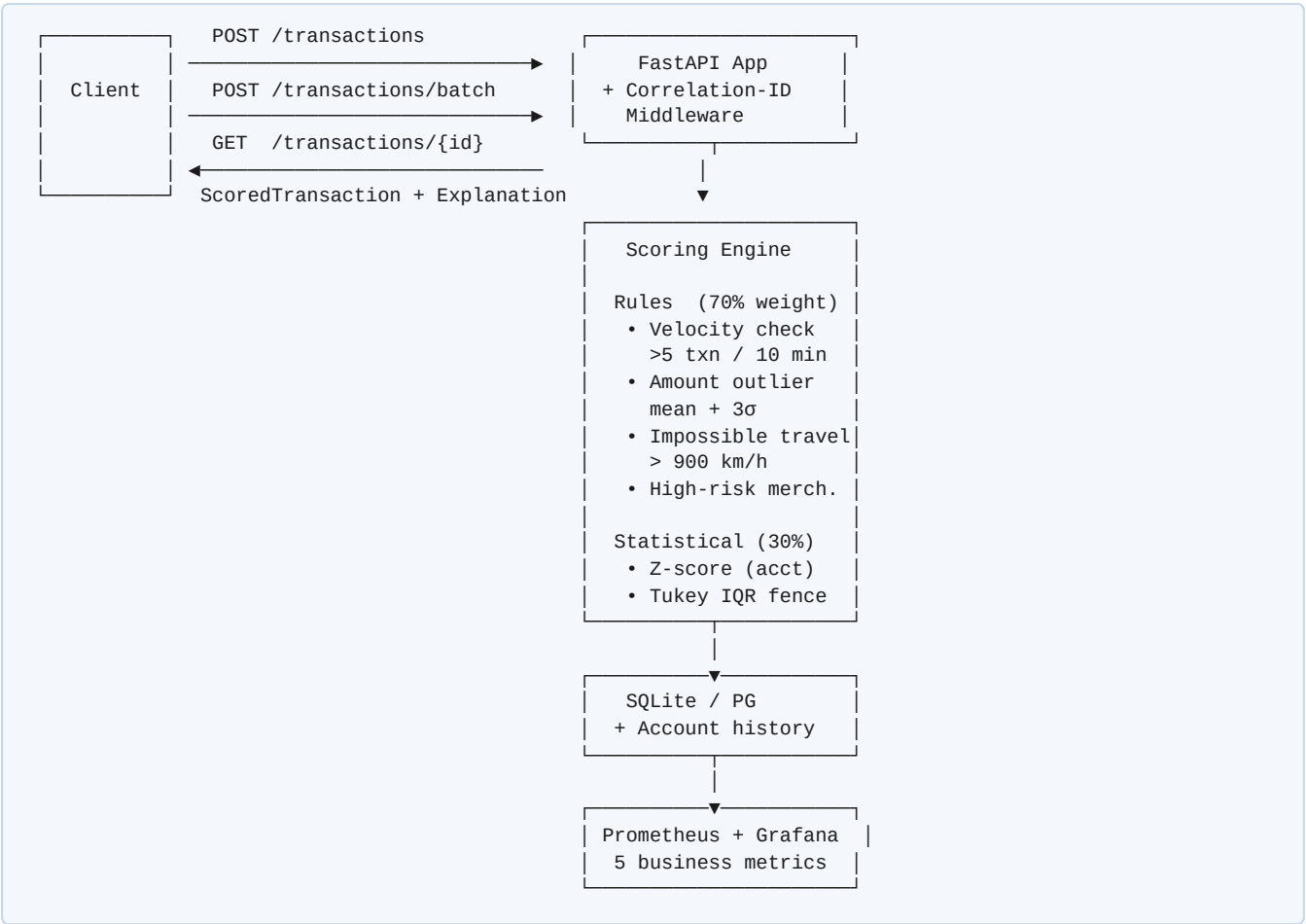
- Drain clustering reduced LLM token spend by treating template extraction as a deterministic pre-processing step — only scored clusters (not every raw log line) ever reach the Anthropic API.
- Confidence-based routing mirrors the production pattern at Mastercard (SENTINEL agent): high-confidence findings auto-escalate to Jira; low-confidence findings surface for human review — keeping the system trustworthy at scale.
- MCP-first design means the same tool can be wired into any MCP-compatible client (Claude Desktop, Claude Code, future VS Code extension) without changing the server code.
- The project is explicitly built to be read by engineers — the ADRs explain not just what was chosen, but what was rejected and why, demonstrating production-level engineering judgment.

This project is a legally-clean, public demonstration of the same log-mining and LLM-RCA patterns applied in production — where a similar system at Mastercard reduced mean time to resolution from hours to minutes across L1/L2 Splunk incident workflows.

TxSentry — Real-Time Transaction Anomaly Scoring

PYTHON · FASTAPI · PROMETHEUS · GRAFANA · POSTGRESQL · EXPLAINABLE AI · FINTECH

Repository: github.com/pavankumarale-hub/txsentry



TxSentry scoring engine — rule-based (70%) + statistical (30%) composite, every decision fully explained

Brief

TxSentry is a FastAPI service that ingests financial transactions — individually or as batch CSV uploads — and scores each for anomalous behavior using a **rule-based + statistical composite scorer**. Every score ships with a full explanation object: which rules fired, what the statistical anomaly score was, and what drove the final risk tier. The system is fully instrumented with **five custom Prometheus business metrics** (beyond generic HTTP stats) and a provisioned Grafana dashboard, demonstrating that production-grade observability means tracking the fraud picture, not just server health.

System Metrics

3 Risk tiers — LOW (0–29) · MEDIUM (30–59) · HIGH (60–100), per-account statistical baseline	70/30 Rule-to-statistical weighting — domain rules provide precision; statistical catches novel anomalies	900 km/h Impossible travel threshold — above commercial flight speed flags geo-sequence fraud
5 Custom Prometheus metrics — risk tier distribution, rule triggers, scoring latency, score histogram, batch quality	Per-acct Z-score baseline computed against each account's own history — not a global threshold	Inline Explanation audit log produced by the same code that scored — zero explanation drift, assertion-safe in tests

Tools Used

Python, FastAPI, uvicorn, SQLAlchemy, SQLite (dev) / PostgreSQL (prod), Prometheus client, Grafana (provisioned dashboard), structlog (structured JSON logging), pytest (unit + integration), Docker Compose, synthetic data generator with documented distribution choices.

TxSentry — Observability & Engineering Decisions

BUSINESS METRICS · EXPLAINABILITY · ALERTING · API DESIGN

Observability — Business-Level Prometheus Metrics

- **txsentry_transactions_scored_total{risk_tier, mode}** — risk tier distribution over time, batch vs. stream split; a sustained HIGH-tier spike triggers PagerDuty.
- **txsentry_rule_triggers_total{rule_name}** — per-rule firing rate; an impossible_travel spike often means bad upstream geo data, not actual fraud.
- **txsentry_scoring_latency_seconds** — scoring engine latency excluding DB I/O; regression detector for algorithmic changes.
- **txsentry_risk_score** — full risk score distribution histogram; a baseline shift signals data drift.
- **txsentry_batch_failed_rows_total** — data quality signal for the CSV ingestion pipeline.

Explainability — Sample API Response

```
{
  "transaction_id": "txn_8a3f...",
  "risk_score": 72.84,
  "risk_tier": "HIGH",
  "explanation": {
    "rules_evaluated": [
      { "rule_name": "impossible_travel", "fired": true, "severity": 1.0,
        "detail": "Implied speed 165200 km/h over 4130 km in 90s > 900 km/h" },
      { "rule_name": "amount_outlier", "fired": true, "severity": 1.0,
        "detail": "Amount 4200.00 exceeds mean+3σ threshold (107.30)" },
      { "rule_name": "high_risk_merchant", "fired": true, "severity": 0.3,
        "detail": "Merchant category 'atm' is elevated-risk" }
    ],
    "statistical": { "method": "z_score+iqr", "score": 1.0,
      "z_score": 226.6, "iqr_outlier": true },
    "dominant_factor": "Rule-based: amount_outlier, impossible_travel"
  }
}
```

Every score ships with a full explanation object — auditable, testable, regulatorily traceable

Key Engineering Decisions

- **Statistical over ML for v1** (ADR-0001) — z-score + IQR is explainable by regulators, needs no training data, and degrades gracefully for new accounts with thin history. ML upgrade path (Isolation Forest → sequence models) documented for v2.
- **Inline explanation audit log** (ADR-0002) — the explanation object is produced by the same code path that computes the score, not a separate "explainer" model. This eliminates explanation drift and makes every rule decision assertion-safe in unit tests.
- **Custom business metrics over generic HTTP metrics** — generic server metrics (HTTP 200/500, latency) tell you the server is alive. Business metrics (risk tier distribution, rule trigger rates) tell you whether the fraud picture is changing.
- **Per-account statistical baselines** — global z-score thresholds would flag anyone who naturally transacts in large amounts. Per-account history makes the scorer adaptive and reduces false positives without sacrificing recall.

Key Insights

- Transaction monitoring systems are judged on explainability as much as accuracy — every HIGH-risk alert that can't be explained to an analyst is a wasted escalation. The explanation object makes TxSentry audit-ready.
- A rule_name=impossible_travel Prometheus spike that correlates with no HIGH-risk tier increase is a data quality signal, not a fraud signal — illustrating why business metrics outperform generic HTTP dashboards for operational intelligence.
- The zero-config SQLite → one-env-var PostgreSQL upgrade path makes the project trivially runnable locally while demonstrating production-readiness awareness.

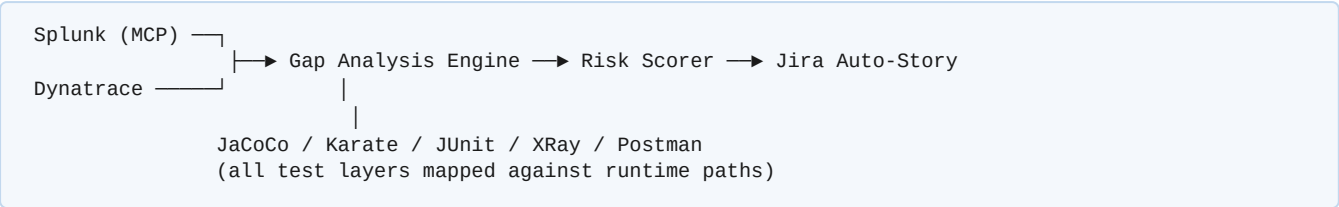
SENTINEL — Production AI Risk Analyzer

PYTHON · SPLUNK MCP · DYNATRACE · JIRA · LLM AGENT DESIGN · MASTERCARD

Brief

SENTINEL is a production Python AI agent built at Mastercard that mines Splunk logs and Dynatrace distributed traces, maps runtime scenarios against all test layers (JaCoCo, Karate, JUnit, XRay, Postman), computes risk scores for uncovered scenarios, and auto-raises Jira stories for coverage gaps — eliminating manual test gap analysis across a team processing **30M+ daily transactions**.

Architecture Highlights



Key Metrics & Impact

- **Zero manual test gap analysis** — SENTINEL automated the full cycle from Splunk log mining to Jira story creation across the team.
- **30M+ daily transactions** — runtime coverage runs against production-scale Splunk data, not sampled or synthetic logs.
- **MTTR: hours → minutes** — LLM-driven RCA agents for L1/L2 Splunk log analysis cut mean time to resolution across incident workflows.
- **MCP-integrated toolchain** — Splunk, Jira, Confluence, and Bitbucket connected via MCP servers, enabling the agent to read, reason, and write across the full engineering toolchain in a single workflow.

AI Agent Library — Team-Wide Engineering Acceleration

LLM AGENTS · PROMPT ENGINEERING · CLAUDE CODE · GITHUB COPILOT · MASTERCARD

Brief

Designed and contributed to a team-wide AI agent library spanning four distinct agent classes, each targeting a high-friction engineering workflow. Collectively these agents reduced rework, improved onboarding speed, and enabled safer, repeatable delivery at 30M+ transaction-per-day scale.

Agent Classes

- **LLM-Driven RCA Agents** — analyze L1/L2 Splunk logs, correlate with Dynatrace traces, and generate plain-language root cause hypotheses with recommended next actions. Reduced MTTR from hours to minutes.
- **LLD Auto-Generation Agents** — given a Jira story and Confluence design context, draft low-level design documents following team architecture standards. Cut design drafting time by **~60%**.
- **PR Review Agents** — enforce consistent standards across logic, UI, and acceptance criteria on every pull request. Reduced review rework and improved onboarding for new engineers.
- **Functional & Integration Test Generation Agents** — generate Karate / JUnit test skeletons from OpenAPI specs and service contracts. Reduced time-to-coverage for new endpoints.

Key Metrics

~60% Reduction in low-level design drafting time via LLD auto-generation agents	hrs → min MTTR improvement via LLM-driven RCA agents on L1/L2 Splunk incidents	4 types Agent classes: RCA, LLD generation, PR review enforcement, test generation
---	--	--

Professional Experience

7+ YEARS — FINTECH · AUTOMOTIVE · DIGITAL HEALTH · AI ENGINEERING

Senior Software Engineer — Technical SME

Mastercard | New York, USA

May 2022 – Present

- Technical SME for consumer-facing financial microservices processing **30M+ daily transactions** for global cardholders.
- Designed and delivered a major service scope extension: 9 new REST endpoints, 3 database tables, Axon event-driven integration, and Spring Scheduler capability.
- Improved core service availability from **99% → 99.9999%** (six nines) through proactive Splunk + PCF log monitoring and systematic root cause resolution.
- Built **SENTINEL** — production AI agent mining Splunk + Dynatrace, mapping runtime coverage gaps, auto-raising Jira stories; eliminated manual test gap analysis.
- Delivered team-wide **AI Agent Library** (RCA, LLD generation, PR review, test generation) — cut design cycle time by ~60%, reduced rework and onboarding time.
- Mentored 5+ engineers on design patterns, Axon event contracts, Pact testing, and multi-environment debugging.

Full Stack Java Developer

Volkswagen | Michigan, USA

June 2021 – May 2022

- Led backend development for a cross-platform mobile application (iOS + Android), acting as middleware between frontend teams and third-party vehicle communication providers.
- Optimized CI/CD pipelines for **40+ microservices** using Jenkins and Docker — reducing release times and improving deployment reliability.
- Engineered serverless architectures using AWS Lambda and API Gateway, reducing operational costs and improving microservice scalability for connected vehicle services.

Full Stack Java Developer

Ford Motor Company (via DMI) | Michigan, USA

Jan 2021 – June 2021

- Built components for Ford's **Machine Learning Operations platform** managing real-time and batch data, feature, and prediction pipelines across the end-to-end model lifecycle.
- Implemented RBAC for developer and manager personas; designed data analysis views for ML model monitoring — demoed to stakeholders and secured production sign-off.
- Optimized ML workflows using TensorFlow; engineered Docker containers reducing deployment time for the MLOps platform.

Full Stack Java Developer

Ontrak | California, USA

Jun 2020 – Jan 2021

- Designed and implemented an inter-service communication pipeline for onboarding new consumers, improving platform scalability and reducing integration time-to-market.
- Engineered a DevSecOps framework integrating security practices into the CI/CD pipeline, enhancing deployment security for a digital health platform.

Software Engineer

Global Systems Network | Hyderabad, India

Jun 2017 – May 2018

- Built a card authorization and transaction management web application for financial issuers with role-based access control.
- Automated full test plan and execution using Jenkins, reducing manual QA effort by **50%** and improving release confidence.

Education

Master of Science, Computer Science

Southern Illinois University Carbondale | Carbondale, IL

GPA: 3.9 / 4.0

Bachelor of Technology, Computer Science Engineering

JNTUH | Hyderabad, India

CLOUD & PLATFORM

- **AWS Cloud Practitioner** — Achieved Cloud Certification (Mastercard engineering team accreditation)
- **Google Cloud — Vertex AI Agent Builder** — Engineered production solutions automating customer interactions
- **Azure** — Applied Azure services in enterprise security and compliance workflows at Mastercard

AI & ENGINEERING

- **LLM Agent Design & Prompt Engineering** — Production experience with Claude API (Anthropic), GPT-4o (OpenAI), Gemini (Google), and Assistants API
- **MCP Server Integration** — Splunk, Jira, Confluence, and Bitbucket MCP integrations deployed in production at Mastercard
- **Claude Code & GitHub Copilot** — Active practitioner; applied to LLD generation, PR review, and test generation workflows

TECHNICAL PROFICIENCIES AT A GLANCE

Backend & Distributed Systems

Java 17/11/8 · Spring Boot · Spring Batch · Spring Security · Axon
SDK · Kafka · Redis · REST / OpenAPI 3.0 · Hibernate · MapStruct · Lombok

Testing & Quality

Pact V3 (CDC) · Testcontainers · JUnit 5 · Karate · JaCoCo · SonarQube · Checkstyle · SpotBugs · Mockito · pytest

Databases

PostgreSQL · MS SQL Server · Oracle · MySQL · Cassandra · MongoDB · Redis · Neo4j · SQLite · DynamoDB · OpenSearch

AI & ML Engineering

Python · FastAPI · Flask · LangChain · Anthropic SDK · MCP Python SDK · PyTorch · TensorFlow · pandas · NumPy · scikit-learn

Cloud, Infra & Observability

AWS (S3, Lambda, DynamoDB, SQS) · GCP · Azure · Docker · Kubernetes · PCF · Prometheus · Grafana · Splunk · Dynatrace · Kibana · Jenkins · GitHub Actions

Frontend & Full Stack

TypeScript · React · Angular · HTML5 · CSS3 · Bootstrap · jQuery · JSP · Streamlit

Contact

Thank you for reviewing my portfolio. I would welcome the opportunity to discuss how I can bring production-scale distributed systems and AI engineering experience to your team.

Name: Pavan Kumar A

Email: pavan88520@gmail.com

Phone: (270) 996-7267

LinkedIn: [linkedin.com/in/pavan-kumar-ba20031b3](https://www.linkedin.com/in/pavan-kumar-ba20031b3)

GitHub: github.com/pavankumarale-hub

Location: Jersey City, NJ 07307